

Institute of Science and Technology

Department of Computer Science and Engineering

Result Processing System

Project Report and Software User Manual

An academic project report describing the design implementation and operation of a role based result processing application

Submitted By

Kazi Saif Hossain Emon

Student ID 22075

Session 2021–2022

Registration No _____

Supervised By

Name _____

Designation _____

Department of Computer Science and Engineering

Course Name and Code _____

Date of Submission _____

ABSTRACT

The Result Processing System (RPS) is a role-based academic web application that connects student registration, a fixed curriculum, session-specific teacher assignment, assessment entry, administrative publication and student result viewing. It is designed for an understandable local workflow in which administrators manage academic setup, teachers work only within assigned courses and sessions, and students view their own published results.

The implementation uses FastAPI, SQLAlchemy and SQLite with a responsive HTML, CSS and JavaScript interface. Academic batches store current and result-processing semesters separately. In the prepared CSE 2021–2022 configuration, the current semester is 7 while results are processed for Semester 6. Core courses are enrolled automatically; Semester 8 elective theory and lab choices remain separate. Marks are validated against the configured component maxima, and totals, grades, semester GPA and CGPA are computed on the server.

The project includes revision-checked marks updates, publication locks, protected deletions, CSV import validation, account provisioning and backup commands. Nine HTTP integration tests and six frontend state tests passed during the documented review. A browser walkthrough verified teacher marks entry, publication and a student GPA/CGPA of 3.50 on fictional demonstration records. The system is suitable for a local academic demonstration; institutional grading approval, retake policies and production deployment remain outside its present scope.

Keywords: Result processing, role-based access control, FastAPI, SQLite, academic session, GPA, CGPA, result publication.

DECLARATION

I, Kazi Saif Hossain Emon, Student ID 22075, declare that this report presents the Result Processing System project and its software operation for academic review in the Department of Computer Science and Engineering at the Institute of Science and Technology.

The report describes the implemented software, its configured rules and the tests performed. External technical references and the supplied formatting reference are acknowledged. Demonstration grades are distinguished from official academic results, and no claim of institutional approval or production certification is made.

Signature _____

Kazi Saif Hossain Emon
Student ID 22075
Session 2021–2022

Date _____

ACKNOWLEDGEMENT

I express my gratitude to the teachers and academic community of the Department of Computer Science and Engineering at the Institute of Science and Technology for the opportunity to study software engineering through a practical academic project.

I acknowledge the official documentation of the technologies used in the Result Processing System, particularly the materials on authentication, relational integrity and concurrency control. These resources support the technical concepts discussed in this report.

I am thankful to my family and well-wishers for their encouragement. I also acknowledge the academic report supplied as a structural reference for organizing this project report and user manual.

Kazi Saif Hossain Emon
Student ID 22075

CONTENTS

Abstract..... i
Declaration..... ii
Acknowledgement..... iii
Contents..... iv–v
List of figures and tables..... vi
Chapter 1 Introduction..... 1
Objectives and scope..... 2
Chapter 2 Literature review..... 3
Technical foundations and comparison..... 4
Chapter 3 Feasibility study..... 5
Chapter 4 Requirement analysis..... 6
Functional and nonfunctional requirements..... 7
Business rules and execution environment..... 8
Chapter 5 System design..... 9
Use case design..... 10
Data flow design..... 11
Database design..... 12
Sequence and state design..... 13
Chapter 6 Implementation and results..... 14
Grading and result calculation..... 15
Login interface..... 16
Administrative dashboard..... 17
Teacher and administrator workspaces..... 18
Administrative result review..... 19
Student results and implementation outcome..... 20

CONTENTS CONTINUED

Chapter 7 Testing and validation..... 21
Test cases and observed evidence..... 22
Chapter 8 Conclusions and future work..... 23
References..... 24
Appendix A Deployment and user manual..... 25
Administrator setup manual..... 26
Teacher assignment and account manual..... 27
Teacher marks entry manual..... 28
Publication and student manual..... 29
CSV import and elective manual..... 30
Backup recovery and troubleshooting..... 31
Appendix B Database dictionary and interface reference..... 32
API and maintenance reference..... 33

LIST OF FIGURES AND TABLES

Figures

Figure 5.1 Layered architecture of the Result Processing System....9
Figure 5.2 Role-specific use cases....10
Figure 5.3 Main academic data flows and logical data stores....11
Figure 5.4 Database entities and principal relationships....12
Figure 5.5 Save review publish and view sequence....13
Figure 6.1 Redesigned login interface....16
Figure 6.2 Administrative dashboard with batch progress....17
Figure 6.3 Teacher marks entry in an assigned session....18
Figure 6.4 Component-level result review before publication....19
Figure 6.5 Student result view with verified GPA and CGPA....20

Tables

Table 1.1 Prepared CSE demonstration configuration....2
Table 2.1 Comparison of result-processing approaches....4
Table 3.1 Development milestones....5
Table 4.1 Role responsibility matrix....6
Table 4.2 Functional requirements....7
Table 4.3 Nonfunctional requirements....7
Table 4.4 Execution environment....8
Table 6.1 Implemented grade thresholds....15
Table 7.1 Automated integration coverage....21
Table 7.2 Representative acceptance and negative tests....22
Table A.1 Teacher assessment limits....28
Table A.2 Common problems and corrective actions....31
Table B.1 Academic registry data dictionary....32
Table B.2 Operational data dictionary....32
Table B.3 Main interface routes....33

CHAPTER 1

INTRODUCTION

1.1 Definition of the system

The Result Processing System (RPS) is a role-based web application for organizing academic records and processing semester results. It brings department and session records, students, teachers, curriculum courses, teaching assignments, assessment marks, publication and student result viewing into one local browser-based workspace. The application is developed using FastAPI, SQLAlchemy and SQLite, with a plain HTML, CSS and JavaScript interface.

1.2 Problem statement

A result-processing workflow involves more than adding marks. Each result must belong to the correct student, course, department, academic session and semester. When separate registers or spreadsheets are maintained without a shared workflow, repeated entry, inconsistent course lists and accidental access to the wrong batch become possible. A teacher also needs a clear boundary between the courses they teach and the records they are permitted to change.

A second problem is the distinction between a student's current semester and the semester whose results are being processed. The prepared CSE 2021–2022 batch is currently in Semester 7, while the intended demonstration processes Semester 6 results. Treating these two values as the same would make past-semester result processing unnecessarily difficult.

1.3 Purpose

The purpose of RPS is to provide a small, understandable academic application in which administrators control academic setup, teachers enter marks within an assigned scope, and students view their own published results. It supports the practical sequence from registering a batch to reviewing and publishing a result, while preserving a clear separation between draft work and student-visible information.

1.4 Report organization

This report presents the project background, relevant technical concepts, feasibility, requirements, design, implementation and test evidence. The deployment and user manual in Appendix A explains the exact operational steps for administrators, teachers and students. Appendix B summarizes the data model and interface contracts needed for maintenance.

Objectives and scope

1.5 Objectives

- Maintain department, session, student, teacher and course records in a consistent relational database.
- Automatically assign the department’s fixed core curriculum to students of a configured academic batch.
- Assign teachers to courses for a specific session and prevent access to other sessions.
- Validate assessment marks, calculate totals and grades, and derive credit-weighted semester GPA and CGPA.
- Separate draft marks from published results and protect saved records against stale edits.
- Provide a clear user interface, a reproducible startup procedure, database backup and a practical user manual.

1.6 Scope of the implemented system

The implemented local application includes account provisioning, temporary passwords, forced first-login password change, role-based access checks, CSV imports for students and teachers, batch settings, fixed-course enrollment, teacher assignment, marks entry and result publication. The prepared curriculum contains 81 CSE courses: 53 core courses, 14 optional theory courses and 14 optional lab courses. Optional choices are made separately in Semester 8.

1.7 Project boundaries

RPS is a local academic demonstration rather than an institution-approved production service. It does not implement retake attempts, replacement of previous grades, automatic email delivery, single sign-on, printable official transcripts or a complete audit-history module. It does not claim that its configured grading policy has been formally approved by the institution. Public hosting requires additional deployment, security, backup and operational review.

1.8 Prepared batch

Table 1.1 Prepared CSE demonstration configuration

Item	Configuration
Department	CSE
Academic session	2021–2022
Registered students	70
Registered teachers	9
Current semester	7
Result-processing semester	6
Fixed core enrollments	3,710 across all eight semesters

CHAPTER 2

LITERATURE REVIEW

2.1 Academic result-processing approaches

Academic results can be maintained using paper registers, spreadsheet workbooks or a structured application. These approaches differ mainly in how they represent relationships, validate inputs, separate responsibilities and control publication. A spreadsheet can calculate a GPA correctly when its formulas and data are maintained carefully; the challenge addressed by RPS is the surrounding workflow, including batch identity, teaching responsibility, access control and review status.

RPS therefore models the academic entities explicitly. A student belongs to a department and session, a course belongs to a department and semester, and a teaching assignment joins a teacher, course and session. This design makes the batch boundary a condition checked by the application rather than a convention stored in a worksheet name.

2.2 Role-based access control

Role-based access control associates permissions with organizational responsibilities. NIST describes RBAC as restricting access according to user roles [3]. RPS applies this concept through three roles: administrator, teacher and student. Role checks alone are not sufficient: a teacher must also match the assigned teacher code, course and session, while a student request must match that student’s reference code.

2.3 Authentication and validation

FastAPI documents password hashing, bearer-token authentication and JWT validation as components of an authenticated application [1]. RPS uses this general approach with password hashes, expiring tokens and database-backed user checks. Input validation is performed on the server as well as in the interface. Browser controls improve usability, but the server remains responsible for rejecting invalid marks, duplicate records and unauthorized changes.

Technical foundations and comparison

2.4 Relational integrity and transactions

A relational database can express dependencies between academic records. SQLite documents foreign keys as a way to enforce that referenced records exist; the feature must be enabled for each connection [2]. RPS enables foreign-key enforcement and groups related writes within transactions. For example, a batch import is validated before insertion so that invalid rows do not leave a partially imported batch.

2.5 Optimistic concurrency

A user may review data that another user subsequently changes. SQLAlchemy's documentation discusses version counters for detecting stale records [4]. RPS implements its own revision-checked conditional SQL updates for marks and publication. It does not configure the SQLAlchemy mapper version counter. The browser submits the revision it displayed; a mismatch returns a conflict instead of silently replacing a newer result.

2.6 Scope comparison

Table 2.1 Comparison of result-processing approaches

Criterion	Separate register or workbook	Implemented RPS
Academic relationships	Maintained through conventions and links	Department, session, course and enrollment records
Teacher access	Depends on document-sharing arrangements	Role plus course and session assignment checks
Publication	Usually a separate distribution step	Explicit draft and published states
Concurrent edits	Depends on the document platform	Marks revision checked before save or publish
Deployment	Document or spreadsheet environment	Local Python application with SQLite
Institutional fit	Highly adaptable manually	Fixed project rules requiring institutional review

The comparison describes the operational scope of this project, not a measured benchmark of commercial products. RPS concentrates on a small, demonstrable workflow and avoids claiming the reporting, scalability or integration capabilities of a full university information system.

CHAPTER 3

FEASIBILITY STUDY

3.1 Operational feasibility

The three roles correspond to familiar academic responsibilities. Administrators prepare records and review results; teachers enter assessment marks; students read published results. Fixed curriculum enrollment removes repeated course assignment for every student. Separate current and processing semesters allow the prepared batch to study Semester 7 while Semester 6 results are processed.

3.2 Technical feasibility

The application runs from one local address and serves both the interface and API. SQLite avoids a separate database-server installation for the demonstration. The backend uses typed validation, relational records and transaction boundaries. The demonstrated workload includes 70 students and 3,710 core-course enrollments, but no controlled load benchmark or large multi-user capacity claim is made.

3.3 Economic feasibility

The project relies on open-source development tools and can be demonstrated on an existing computer. The principal practical costs are development time, data preparation, testing and maintenance. Public hosting would introduce additional operational costs, including a hosting environment, backups, secure access and administration. The feasibility assessment is qualitative; institutional deployment would require a formal cost and benefit review.

3.4 Adoption and schedule feasibility

The interface uses ordinary forms and role-specific navigation. Name-and-code selectors reduce the need to memorize identifiers, and temporary-password messages support first-time users. Development proceeded through repeated requirement clarification, implementation and testing. The milestones below summarize the implemented development phases.

Table 3.1 Development milestones

Milestone	Evidence
Academic model and setup	Ten application tables and startup commands
Complete role workflow	Administrator, teacher and student integration test
Result safeguards	Revision checks, publication locks and GPA tests
Frontend refinement	Responsive interface and six frontend state tests
Documentation	Project report and deployment/user manual

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Development methodology

An iterative development approach was used because operational requirements were clarified while the application was being exercised. Initial code review identified broken pages and incomplete database setup. Subsequent iterations added the complete role workflow, batch-specific teacher assignment, fixed enrollment, safer marks editing and clearer frontend screens. Each meaningful backend correction was checked using an isolated test database rather than the real academic records.

4.2 Actors and responsibilities

Table 4.1 Role responsibility matrix

Operation	Admin	Teacher	Student
Manage academic records	Yes	No	No
Create or reset login accounts	Yes	No	No
Set batch semesters and assign teachers	Yes	No	No
Enter assessment marks	No	Assigned scope only	No
Review and publish marks	Yes	No	No
View semester results	All students	No student-results endpoint	Own published results
Change own password	Yes	Yes	Yes

4.3 Functional requirement groups

The system must identify the user, establish the permitted academic scope and then apply the rules for the requested operation. Administrative maintenance, curriculum enrollment, assessment, publication and result viewing form the main functional groups. The distinction between authentication, role authorization and academic ownership is maintained in both the design and tests.

4.4 Data privacy requirements

Password hashes must not be returned by account-listing endpoints. A student must not retrieve another student’s results. A teacher must not retrieve the roster or marks of a course/session they are not assigned to. Demonstration screenshots use fictional records rather than publishing the supplied students’ personal email addresses.

Functional and nonfunctional requirements

Table 4.2 Functional requirements

ID	Requirement	Acceptance condition
FR01	Authenticate users	Valid credentials return an expiring token
FR02	Provision accounts	Temporary password displayed after creation/reset
FR03	Maintain academic registries	Validated create, edit and protected delete
FR04	Import students and teachers	Preview first; invalid batch writes nothing
FR05	Configure academic batches	Processing semester does not exceed current semester
FR06	Apply fixed curriculum	Core enrollment repeats without duplicates
FR07	Assign teachers	Assignment includes teacher, course and session
FR08	Save assessment marks	Four bounded values and revision checks
FR09	Publish reviewed results	Only admin can change publication state
FR10	Calculate results	Credit-weighted GPA/CGPA with pending exclusions
FR11	Manage Semester 8 electives	At most one optional theory and one lab
FR12	Back up local data	SQLite backup saved without changing records

Table 4.3 Nonfunctional requirements

Quality	Implemented measure
Security	Role/ownership checks, password hashes and token expiration
Integrity	Foreign keys, unique constraints and transaction boundaries
Usability	Role-specific navigation, search and clear confirmation controls
Reliability	Conflicts rejected instead of overwriting newer marks
Maintainability	Separated models, database setup, service logic and tests
Portability	Local browser interface and documented Python startup
Recoverability	Backup command and manual restoration procedure

Business rules and execution environment

4.5 Core business rules

- A configured batch is identified by department and academic session. Current semester and processing semester are separate values.
- All fixed core courses are enrolled automatically, including future-semester core courses. A future enrollment does not create a result.
- Teacher permissions are restricted by teacher code, course code and session. Future-semester marks remain locked until the batch reaches that semester.
- Assessment maxima are assignment 10, class test 20, midterm 40 and final 80. All four values are required and allow at most two decimal places.
- Students see published results only. Published marks must be unpublished by an administrator before teacher revision.
- Saved results protect relevant academic data: credits cannot change after marks exist, and course type cannot change after enrollment.

4.6 Software and hardware environment

Table 4.4 Execution environment

Component	Project configuration
Backend	Python, FastAPI 0.141.1 and Uvicorn 0.53.0
Persistence	SQLAlchemy 2.1.4 and SQLite
Validation and security	Pydantic 2.13.5, PyJWT 2.15.1 and pwdlib 0.3.1
Frontend	HTML, CSS and JavaScript without a frontend build step
Local address	http://127.0.0.1:8001
Practical equipment	A computer with Python, a modern browser and available disk space

These are the pinned project dependencies, not a claim that they are the latest releases. The supplied development environment uses a Python virtual environment. Hardware figures are not presented as benchmarked minimum requirements; the application should be checked on the actual demonstration computer before submission.

CHAPTER 5

SYSTEM DESIGN

5.1 Architectural design

RPS uses a browser interface, a FastAPI application layer and a relational persistence layer. The frontend calls the API from the same origin and does not open the SQLite database directly. Authentication and academic authorization are checked before business logic executes. SQLAlchemy sessions provide the boundary through which validated changes reach the database.

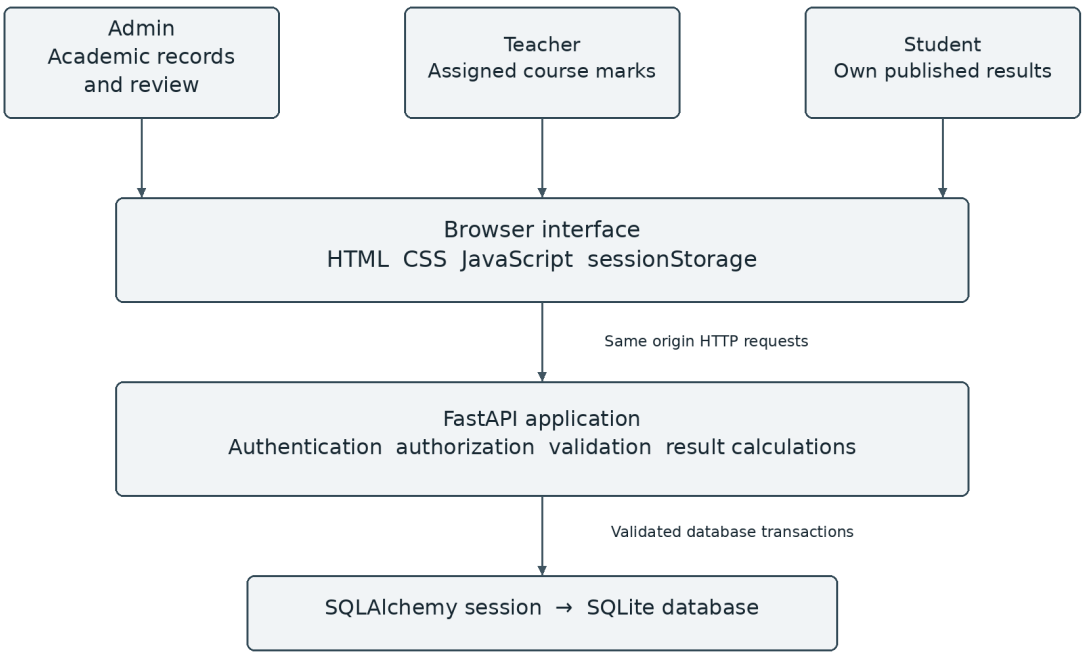


Figure 5.1 Layered architecture of the Result Processing System

5.2 Input and output design

Input forms use labels, bounded numeric fields, selection lists and human-readable errors. Names and public codes appear together when choosing a teacher or account owner. Outputs include record lists, academic-batch summaries, draft marks, publication status and student semester results. Generated totals and grade points are calculated by the server rather than trusted from browser input.

Use case design

5.3 Use case diagram

The administrator establishes the academic workspace and controls publication. Teachers act within assigned offerings, and students read their own results. Password changes are available to every authenticated role; account creation remains an administrative operation.

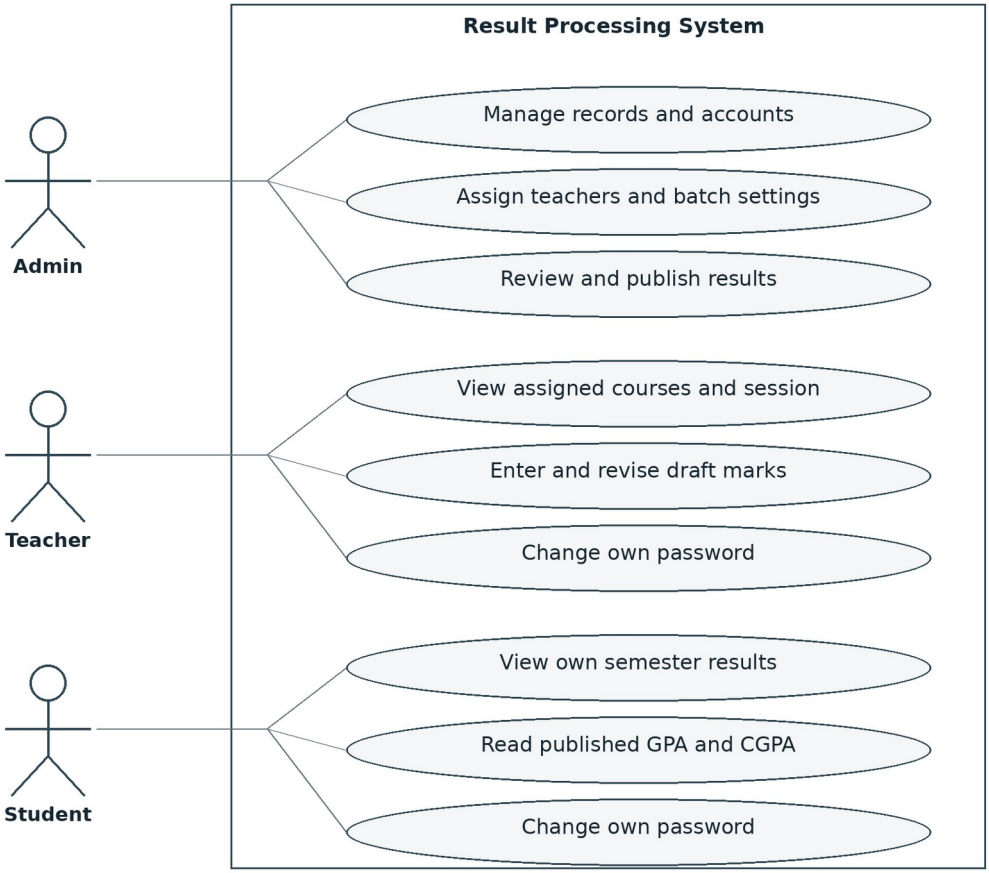


Figure 5.2 Role-specific use cases

A successful login does not grant every use case. Every protected operation still evaluates the required role and, where relevant, the session and owner of the requested record.

Data flow design

5.4 Data flow diagram

Academic setup, marks entry and published-result viewing exchange data through the application. Administrative publication changes the state of an existing marks record; it does not create a separate copy of a result. A student query therefore filters the same underlying records by publication status.

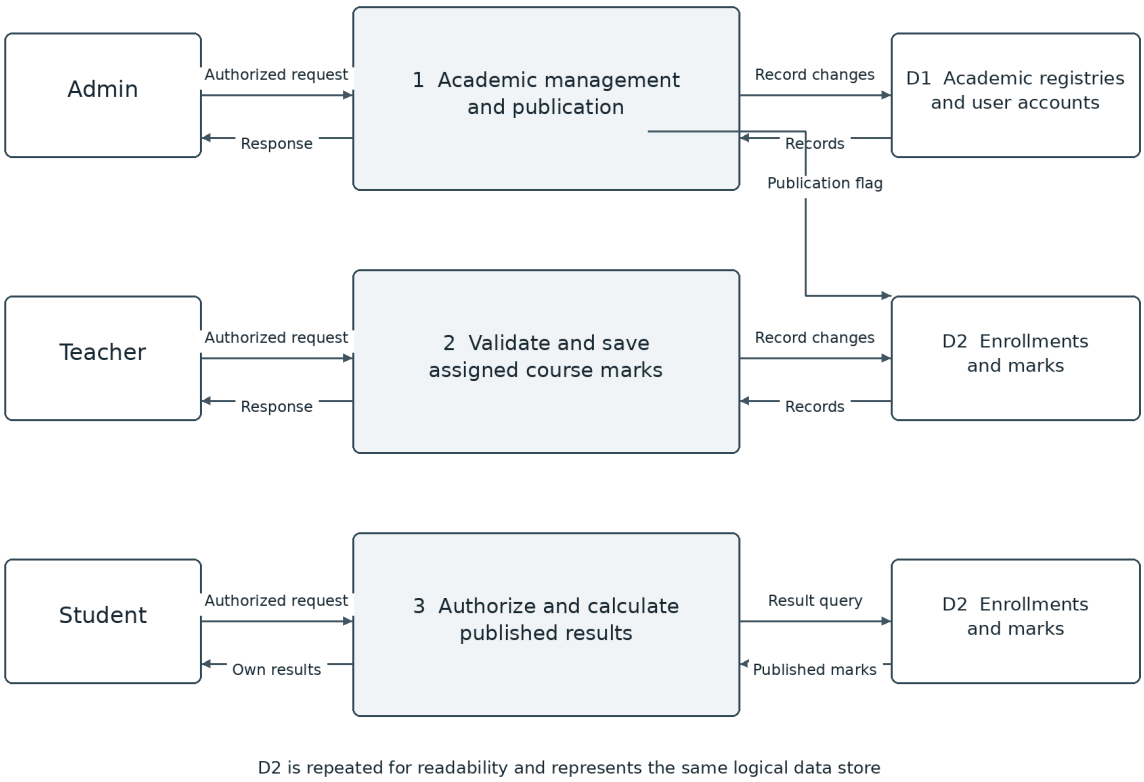


Figure 5.3 Main academic data flows and logical data stores

5.5 Transaction boundaries

CSV import and fixed enrollment are grouped with their associated record changes. Preview operations return validation results without saving. Optional-course removal is blocked when saved marks exist. Deletion checks use conditional database operations to avoid removing a result that was saved while an administrative request was in progress.

Database design

5.6 Entity relationships

The schema contains ten tables. Departments and academic sessions identify batches; students and teachers are registered separately from login accounts. Courses form the static curriculum. Enrollments connect students to courses, and marks connect to a unique enrollment. Appendix B lists the main fields and constraints.

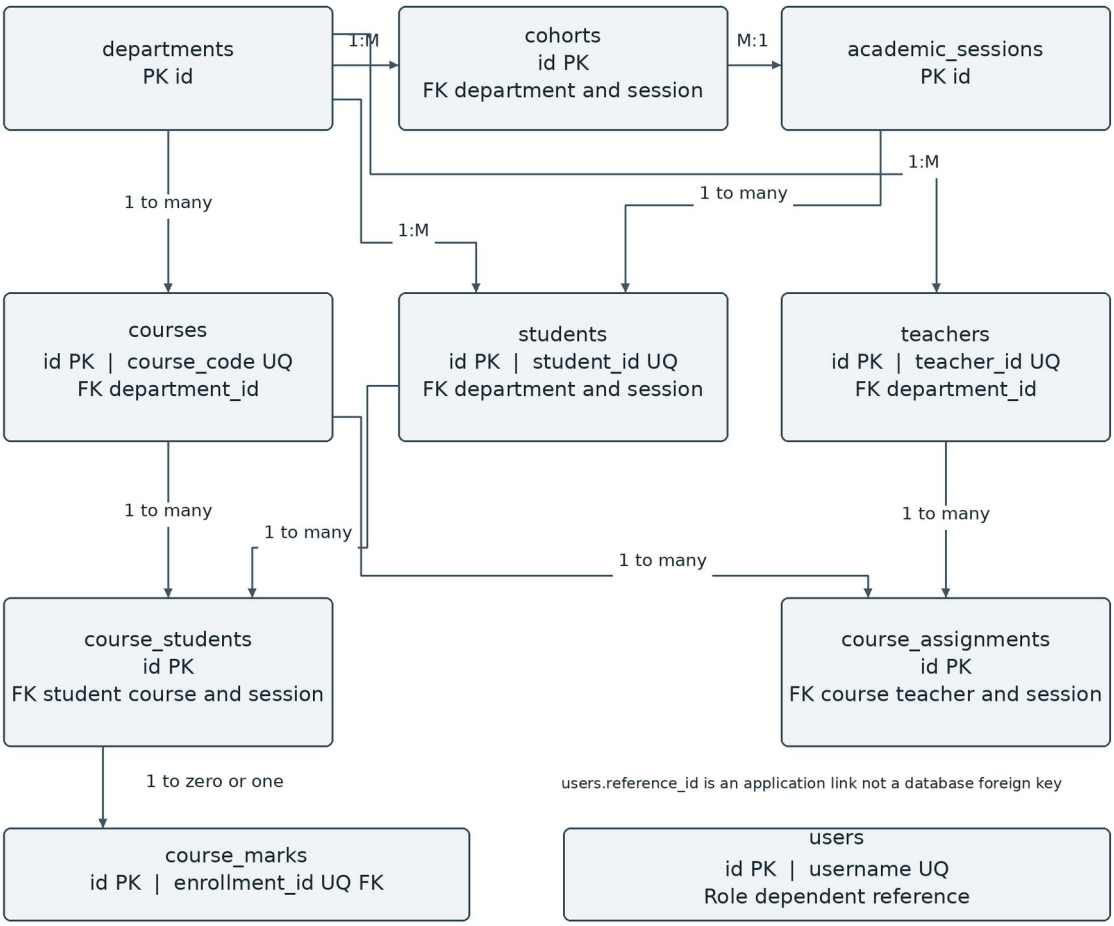


Figure 5.4 Database entities and principal relationships

The user reference code is validated by application logic and is not a database foreign key because it can identify either a teacher or a student. Enrollment stores session and semester explicitly; the API keeps these values aligned with the student and course. A course may have several teachers for a session.

Sequence and state design

5.7 Result processing sequence

Marks entry, administrative review and student viewing are separate actions. A revision number connects what a user reviewed with what the database is asked to modify. Publication increments the revision, so an older teacher screen cannot silently save over a publication decision.

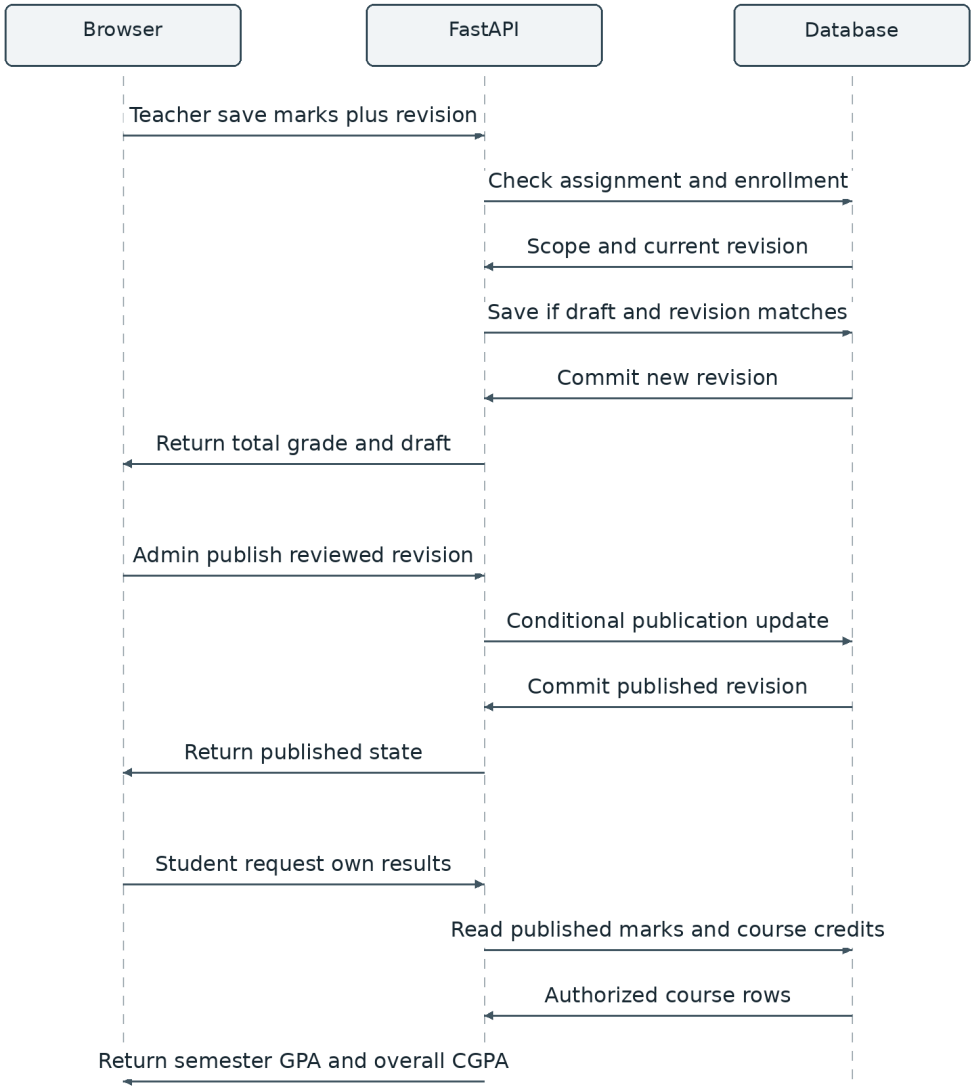


Figure 5.5 Save review publish and view sequence

The operational state changes are: no saved marks → draft → published. Unpublishing returns a record to draft. Each successful save or publication change advances the revision; a stale request receives a conflict and must reload.

CHAPTER 6

IMPLEMENTATION AND RESULTS

6.1 Application structure

```
RPS/
  Backend/
    app/main.py              API and business rules
    app/models/              Academic and account models
    app/db/                  Connection and schema upgrades
    app/services/curriculum.py  Fixed enrollment synchronization
    app/routes/imports.py    CSV validation and import
    tests/test_system.py     HTTP integration tests
    manage.py                Initialization admin setup backup
    requirements.txt         Pinned backend dependencies
  Frontend/
    index.html  app.js  style.css
    tests/state.test.cjs      Frontend state tests
  start.sh                   Local startup helper
```

6.2 Database initialization and migration

The local database is stored separately at Backend/rps.sqlite3. The application deliberately does not reuse the original DATABASE_URL setting. Startup initializes the schema, applies supported local upgrades and synchronizes configured core enrollments. A legacy assignment without a session mapping is not guessed: startup stops with an explanatory error rather than assigning it to every session.

6.3 Account lifecycle

The first administrator is created through an interactive local command. Other accounts are created from the Accounts page. A generated temporary password is shown immediately after creation or reset and must be shared privately. First-time users change that password before accessing academic pages. Password changes and resets invalidate existing tokens through a token-version check.

Grading and result calculation

6.4 Assessment total and grade mapping

For each course, total marks equal assignment plus class test plus midterm plus final. The total is out of 150. Grade thresholds are percentages of that total; for example, 120 out of 150 is 80 percent and maps to A+. All courses currently use the same assessment structure, including lab and project courses. An alternative institutional structure requires a deliberate policy change before real use.

Table 6.1 Implemented grade thresholds

Minimum percentage	Minimum total out of 150	Grade	GP
80	120.00	A+	4.00
75	112.50	A	3.75
70	105.00	A−	3.50
65	97.50	B+	3.25
60	90.00	B	3.00
55	82.50	B−	2.75
50	75.00	C+	2.50
45	67.50	C	2.25
40	60.00	D	2.00
Below 40	Below 60.00	F	0.00

6.5 GPA and CGPA

Semester GPA = sum of course credit × grade point for eligible courses, divided by their total credits. CGPA applies the same calculation across all semesters, even when one semester is selected for viewing. Student calculations use published marks only; administrative previews include saved draft marks. Failed eligible courses count with zero grade points. Pending courses are excluded and the semester is marked Incomplete.

Decimal arithmetic avoids avoidable floating-point artifacts. A verified example with two three-credit courses having grade points 4.00 and 3.00 gives (3 × 4.00 + 3 × 3.00) ÷ 6 = 3.50. No retake or grade-replacement rule is implemented.

Login interface

6.6 Login interface and first access

The login page provides one entry point for all roles. The server determines the correct workspace after authenticating the user. The page identifies the academic portal, provides clearly labelled inputs and displays the developer credit. The screenshot below uses an empty sign-in form.

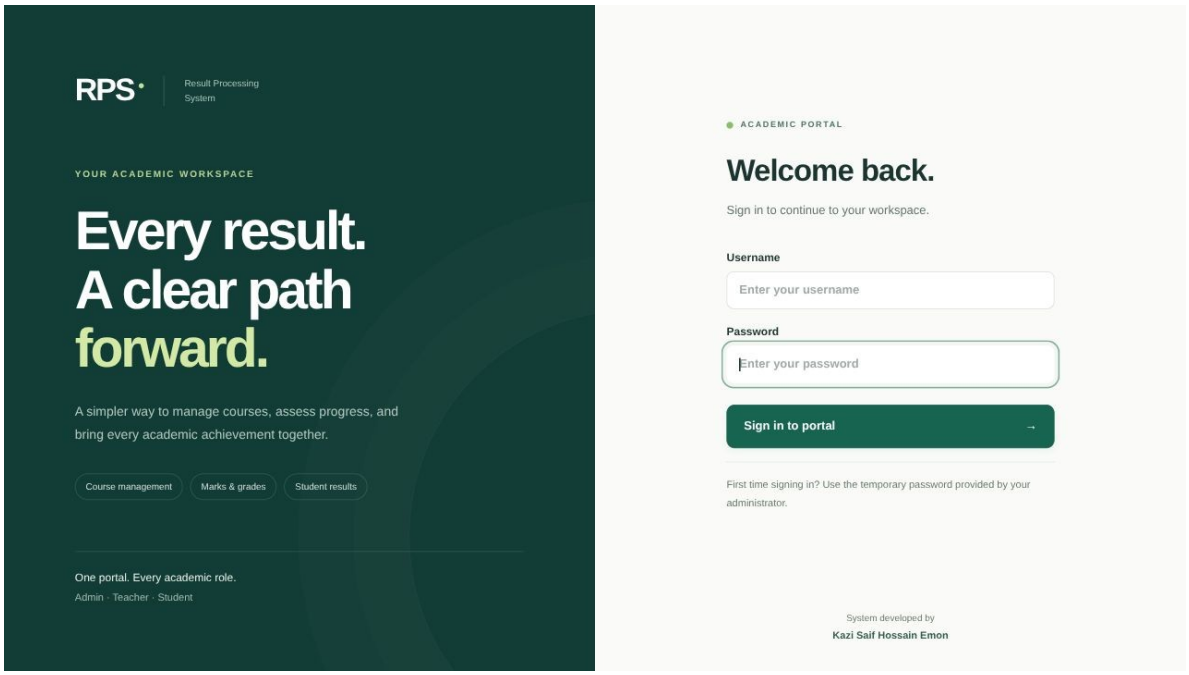


Figure 6.1 Redesigned login interface

6.7 Interface conventions

The interface uses dark green navigation, restrained cards, consistent controls and readable feedback. Role-specific navigation keeps unrelated operations out of each workspace. The footer displays “System developed by Kazi Saif Hossain Emon”. A responsive layout rearranges the navigation and cards for narrower screens; wide record tables remain horizontally scrollable when necessary.

For account recovery, administrators generate a new temporary password rather than retrieving the old password. Plaintext passwords are not stored in the account table, and the interface does not display existing password hashes.

Administrative dashboard

6.8 Dashboard and batch summary

The dashboard summarizes students, teachers, curriculum courses and course registrations. Batch progress displays the current semester separately from the result-processing semester. Interface illustrations in this chapter use fictional review records and sample grades, not official student results.

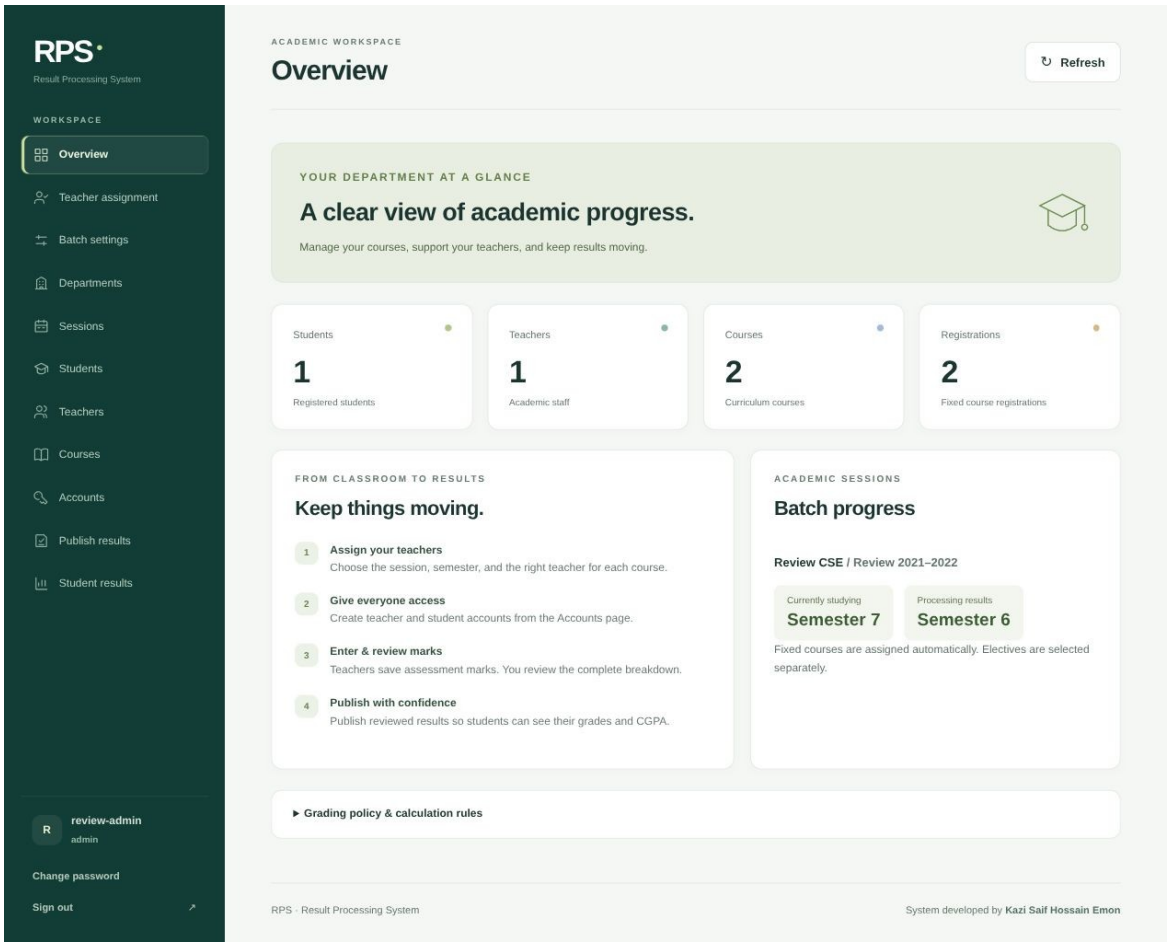


Figure 6.2 Administrative dashboard with batch progress

Teacher assignment and Batch settings are separate, visible navigation items. The removed Assignments and Enrollments menu pages are not required for the normal fixed-curriculum workflow.

Teacher and administrator workspaces

6.9 Teacher assignment and marks entry

The administrator selects a department, session and semester, then chooses a teacher beside a course card. Both the teacher's name and code are visible. The teacher's own workspace lists only assigned offerings, distinguishes sessions and provides searchable student cards with all four assessment fields and a Save marks button.

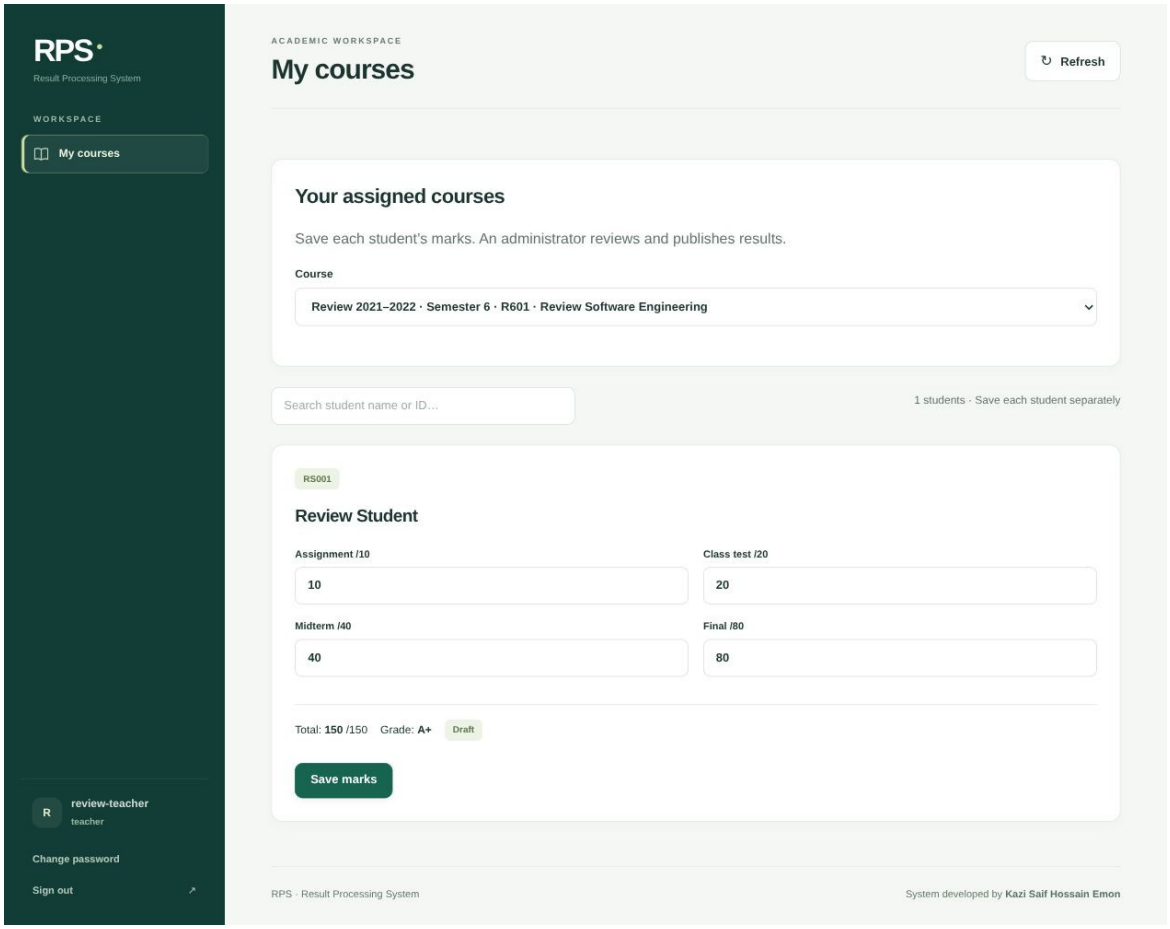


Figure 6.3 Teacher marks entry in an assigned session

Administrative result review

6.10 Administrative review

The Publish results page shows each saved result with its four assessment components, total, grade and grade point. Department, session, semester and text search narrow the list. An in-page confirmation is required before publication. If the record changed after review, publication is rejected and the administrator must reload the latest marks.

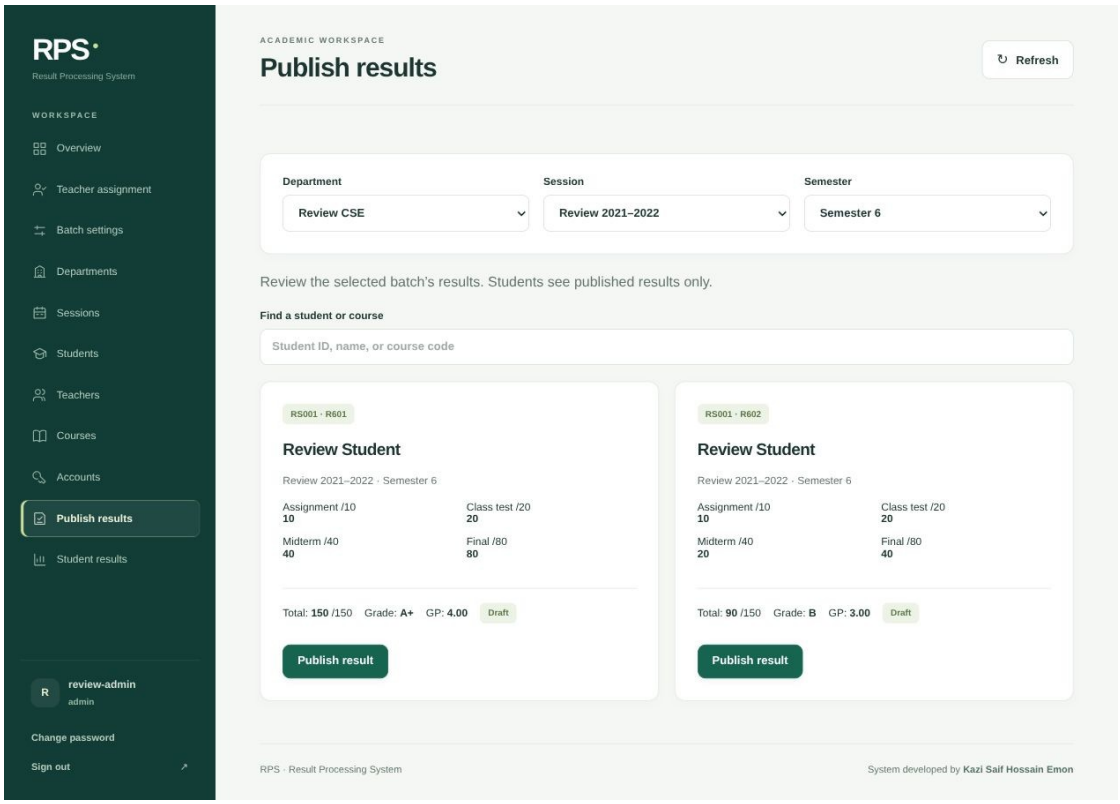


Figure 6.4 Component-level result review before publication

Student results and implementation outcome

6.11 Student result viewing

A student’s result page defaults to that student’s batch processing semester. It shows the student identity, current and processing semesters, overall CGPA and semester courses. Unpublished components are not returned to the student. The same backend ownership check prevents a modified URL from revealing a different student’s results.

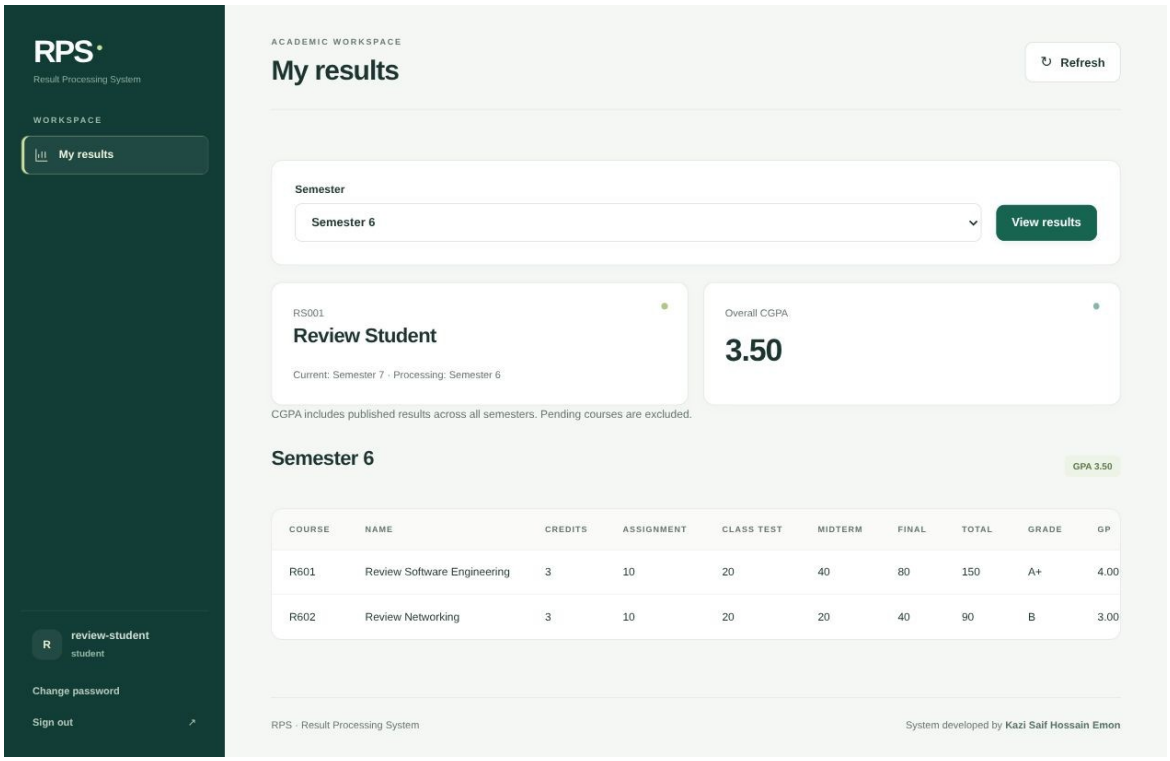


Figure 6.5 Student result view with verified GPA and CGPA

6.12 Implemented outcome

The demonstration completed a full sequence: a teacher saved two fictional course results, an administrator published them, and the student viewed the correct 3.50 GPA and CGPA. The prepared real-data workspace is separate from that review fixture. Successful sample operation is evidence of the tested workflow, not an institutional acceptance certificate or a high-concurrency performance benchmark.

CHAPTER 7

TESTING AND VALIDATION

7.1 Test strategy

Testing combines Python HTTP integration tests, JavaScript state tests and browser walkthroughs. The integration suite starts a temporary server with a separate SQLite database. It provisions synthetic academic records and exercises the actual HTTP endpoints. Frontend state tests run without a browser to verify confirmation, session and unsaved-change logic. Browser walkthroughs check that a user can complete the visible workflow.

Table 7.1 Automated integration coverage

Test group	Verified behavior
Complete journey and permissions	Login change password assign save publish view own results
Bulk CSV import	Preview atomic failure duplicate rows and file limits
Batch assignment and enrollment	Session isolation department/semester matching and repeat sync
Fixed curriculum	New students/imports inherit core courses; optional choices excluded
Database upgrade	Existing enrollments and marks preserved; repeat initialization safe
Marks revision and record protection	Concurrent saves stale publication precision and future locks
Optional enrollment	One theory and one lab; saved marks block removal
Grade boundaries	A+ A D and F threshold checks
Login rate limit	Excess repeated sign-in attempts receive rate-limit response

7.2 Execution result

The final regression run completed nine integration tests and six frontend state tests successfully. Test assertions cover invalid as well as valid requests; a rejected unauthorized operation is a successful security test. The real database also passed SQLite integrity and foreign-key checks. The test count describes the implemented suite and does not imply that every possible deployment or institutional grading policy has been tested.

Test cases and observed evidence

Table 7.2 Representative acceptance and negative tests

ID	Scenario	Expected and observed result
T01	Valid teacher saves 150 marks	Total 150 and grade A+
T02	Teacher queries an unassigned session	403 access denied
T03	Student queries another student	403 access denied
T04	Mark exceeds maximum or is nonfinite	422 validation error
T05	Two edits use the same revision	One save succeeds; stale save returns 409
T06	Admin publishes an outdated review	409 conflict and review required
T07	Teacher edits a published result	409 locked result
T08	Core synchronization repeated	No duplicate enrollments
T09	Import contains one invalid email	No records inserted from the batch
T10	Two equal-credit GP values 4 and 3	GPA and CGPA 3.50
T11	Optional removal includes saved marks	Removal blocked; records preserved
T12	Current semester 7 future course 8	Marks entry blocked until semester update

7.3 Frontend checks

Six state tests verify in-page Confirm/Cancel, preservation of unsaved marks after cancellation, forced sign-out cleanup, handling of stale unauthorized responses, readable errors for invalid server responses, and keeping the CSV preview dialog open before commit. Browser checks verified the responsive dashboard, teacher input cards, review confirmation and student result presentation.

7.4 Evaluation limits

No formal usability study, production penetration test or load benchmark was conducted. Public hosting and official academic deployment remain separate activities. The configured assessment weights and grade thresholds must be confirmed before the system is used to issue official results.

CHAPTER 8

CONCLUSIONS AND FUTURE WORK

8.1 Conclusion

RPS provides a working local path from academic setup to student result viewing. Its main contribution is the explicit connection between department, session, curriculum, teaching responsibility and publication state. Fixed core enrollment removes repetitive student-course assignment, while separate current and processing semesters support delayed result processing for a batch that has already progressed.

The implemented checks address practical failure modes: invalid marks, access to another session, stale edits, publication of an outdated review, accidental changes to credits after marks exist, and deletion of enrollments that have saved results. The user interface presents these rules through ordinary forms, selectors, student cards and confirmation controls.

8.2 Limitations

- Assessment weights are fixed at 10, 20, 40 and 80 for all course types.
- Retakes, repeated attempts and grade replacement are not modeled.
- Account credentials must be shared manually; there is no automated email or SSO.
- Official transcript generation and downloadable result sheets are not implemented.
- The local application has process-local login limits, no complete audit-history module and no production hosting configuration.

8.3 Future development

Useful extensions include institution-approved assessment policies per course type, retake handling, a verifiable result-change audit trail, student-specific elective planning, printable transcripts, notification delivery and a reviewed production deployment. These should be added only after requirements are confirmed and the existing grade calculations and access controls remain covered by tests.

8.4 Completion status

The documented local software workflow is implemented and regression-tested. Final academic demonstration still requires administrator-selected teaching assignments, teacher/student accounts and approved demonstration marks. Production readiness requires separate institutional and operational acceptance.

References

[1] FastAPI. OAuth2 with Password and hashing Bearer with JWT tokens. Official documentation. https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/ (accessed 3 November 2026).

[2] SQLite. SQLite Foreign Key Support. Official documentation. https://www.sqlite.org/foreignkeys.html (accessed 3 November 2026).

[3] National Institute of Standards and Technology. Role based access control glossary. https://csrc.nist.gov/glossary/term/role_based_access_control (accessed 3 November 2026).

[4] SQLAlchemy. Configuring a Version Counter. SQLAlchemy 2.1 documentation. https://docs.sqlalchemy.org/en/21/orm/versioning.html (accessed 3 November 2026).

[5] Result Processing System project source. Backend/app/main.py, Backend/app/models, Backend/app/db and Backend/app/services/curriculum.py. Version 2.1.0, reviewed 3 November 2026.

[6] Result Processing System project documentation and tests. README.md, Backend/tests/test_system.py and Frontend/tests/state.test.cjs. Reviewed 3 November 2026.

[7] CSE curriculum and academic roster supplied for this project. Department CSE, academic session 2021–2022. Used as project configuration data; institutional policy approval is not implied.

[8] Omar Faruk. TaskFlow Role Based Project Management Workspace. Institute of Science and Technology project report, 2026. User-supplied formatting and chapter-structure reference.

APPENDIX A

DEPLOYMENT AND USER MANUAL

A.1 Before starting

The application runs on the demonstration computer. Keep the server terminal open while using the portal. The address 127.0.0.1 refers to the same computer; entering that address on a different phone or laptop does not connect to the demonstration computer. Internet hosting is not configured by the local startup procedure.

A.2 Start on Linux or macOS

- Open a terminal in the RPS project folder.
- Run ./start.sh. On first use it creates a virtual environment if needed, installs missing project dependencies and initializes the database.
- If no administrator exists, choose and confirm an admin password containing 12–128 characters. Password entry is hidden while typing.
- Open http://127.0.0.1:8001 in a browser. Keep the terminal open. Press Ctrl+C in that terminal only when you want to stop the server.

```
cd /path/to/RPS
./start.sh
```

A.3 Manual setup and Windows

From the Backend folder, create a virtual environment. Activate it using source .venv/bin/activate on Linux/macOS, or .venv\Scripts\activate in Windows Command Prompt. Then install the requirements and initialize the application.

```
python -m venv .venv
python -m pip install -r requirements.txt
python manage.py init
python manage.py create-admin
python -m uvicorn app.main:app --host 127.0.0.1 --port 8001
```

Do not create a second admin with an existing username. If an administrator already exists, use its chosen password. An extra local administrator can be created with manage.py create-admin --username another-admin. Keep administrative access private.

Administrator setup manual

A.4 Sign in as administrator

- Open the portal and enter the administrator username and chosen password.
- Use Overview to check record totals and the academic batch summary.
- Open Departments and Sessions to add the required department and academic session if they are not already present.
- Add students, teachers and courses. Codes should be unique and use letters, digits, hyphens or underscores. Do not enter a fabricated email when the correct email is unknown.

A.5 Configure the academic batch

- Open Batch settings.
- Select the department and session.
- Enter the current semester and result-processing semester. The result-processing semester cannot be later than the current semester.
- For the prepared CSE 2021–2022 batch, keep Current semester 7 and Result processing semester 6.
- Select Save batch settings. Fixed core enrollments are synchronized automatically.

A.6 Understand fixed enrollment

All 53 core courses in the configured CSE curriculum are attached to each registered student in that batch. With 70 students this produces 3,710 fixed registrations across all semesters. Semester 6 contains seven core courses and therefore 490 student-course registrations. Registration is not a saved grade: future courses and unsaved courses remain pending.

You do not need a separate Assignments or Enrollments menu to perform the normal workflow. Teacher assignment is controlled from its own page; fixed student enrollment is automatic. Semester 8 electives are chosen separately.

Teacher assignment and account manual

A.7 Assign a teacher to a course

- Open Teacher assignment.
- Select CSE, session 2021–2022 and Semester 6, or another intended combination.
- Find the course card by course code and title.
- Choose the teacher using the displayed teacher ID and full name.
- Select Assign teacher. The assigned name appears on the course card.
- To change teaching responsibility, remove the old assignment and assign the correct teacher. Removing an assignment changes access; it does not delete stored marks.

A.8 Create a teacher or student login

- Open Accounts and select Add accounts.
- Choose Teacher or Student as the role.
- Select the registered person by name and code. People who already have an account are excluded from this selector.
- Check the suggested username or enter another valid username, then select Save.
- Copy the displayed username and temporary password immediately. Share each account privately with its owner.
- If the temporary password is lost, use Reset password beside that account and confirm. A new temporary password replaces the old one and existing logins are invalidated.

A.9 Important password rules

Adding a teacher or student record does not automatically create a login account. The first admin password is chosen during local setup; generated teacher and student passwords appear only after account creation or reset. Existing plaintext passwords cannot be retrieved from the database. A user must change a temporary password before continuing to academic pages.

Teacher marks entry manual

A.10 First teacher login

- Sign out of the admin account or use a separate browser profile.
- Enter the teacher username and temporary password supplied by the administrator.
- Choose Change password, enter the current temporary password and confirm a new 12–128 character password.
- Sign in again with the new password. My courses opens the teacher workspace.

A.11 Enter marks

- Choose the assigned session and course. The processing semester is preferred when a matching assignment is available.
- Use student search to find a name or student ID.
- Enter assignment, class-test, midterm and final marks. Use 0 for an actual zero; do not leave a component blank.
- Select Save marks for that student. Confirm the returned total and grade.
- Repeat for the remaining students. Each student is saved separately; entering values alone does not save them.

Table A.1 Teacher assessment limits

Assessment	Allowed range
Assignment	0–10
Class test	0–20
Midterm	0–40
Final examination	0–80
Precision	At most two decimal places

A.12 Correct and protect draft marks

Draft marks can be edited and saved again. If another teacher edited the result after the page loaded, the save is rejected; reload and review before re-entering the intended change. Unsaved edits require confirmation before leaving or switching courses. Published results are locked, and a future-semester course remains locked until the batch reaches that semester.

Publication and student manual

A.13 Review and publish as administrator

- Sign in as admin and open Publish results.
- Choose the department, session and semester. Use student or course search when needed.
- Review all four assessment components, total, grade and grade point on the result card.
- Select Publish result, read the confirmation and select Confirm.
- If the result changed since you reviewed it, refresh and review the new values before publishing.
- To allow a correction, select Unpublish. Ask the teacher to correct and save the draft, then review and publish again.

A.14 View results as a student

- Sign in with the student account provided by the administrator. Change the temporary password on first use and sign in again.
- Open My results. The student’s configured processing semester is selected by default.
- Select a semester and choose View results when changing the selection.
- Read the course marks, grade, grade point, semester GPA and overall CGPA. Only published marks are visible.
- A dash or Pending indicates an unavailable published result. Incomplete indicates that not all enrolled courses in that semester have eligible grades.

A.15 Interpreting GPA correctly

A failed published course contributes its credits and zero grade points. A pending course contributes neither credits nor grade points to the displayed calculation. CGPA includes eligible results across all semesters, not just the selected semester. Therefore a displayed GPA for an incomplete semester should not be treated as a final official semester result.

CSV import and elective manual

A.16 Bulk student and teacher import

- Open Students or Teachers and select Import CSV.
- Download the template and replace the example row with actual records. Save the file as CSV UTF-8.
- For students use the exact columns student_id,name,email. For teachers use teacher_id,name,email.
- Select the batch department and, for students, the session. Use a separate file for each department/session combination.
- Choose a file smaller than 1 MB with at most 1,000 records, then select Preview CSV.
- Correct every reported error. A duplicate code, duplicate email or invalid row blocks the whole batch.
- Review the preview and select Import records to commit. Account creation remains a separate step.

```
student_id,name,email
S001,Example Student,student@example.com

teacher_id,name,email
T001,Example Teacher,teacher@example.com
```

A.17 Semester 8 optional courses

In Teacher assignment, select Semester 8 and choose only the options actually taken by the batch. The system allows at most one optional theory course and one optional lab per student. Fixed core courses remain automatic. Saving an optional batch selection affects every student of the selected department/session, so confirm the intended batch before committing.

To replace an elective, remove its optional enrollment from the batch first. The interface previews the number affected. If any selected enrollment already has saved marks, removal is blocked to preserve the result. Student-specific academic exceptions require administrative review rather than inventing a retake or replacement policy.

Backup recovery and troubleshooting

A.18 Back up the database

```
cd Backend
.venv/bin/python manage.py backup
```

On Windows, use .venv\Scripts\python manage.py backup. Backups are stored under Backend/backups. Keep a copy outside the working computer when appropriate and protect it because it contains personal records and password hashes. To restore, stop the server, preserve the current database, replace Backend/rps.sqlite3 with the chosen backup and restart. Restoration is a manual operation to prevent accidental replacement.

Table A.2 Common problems and corrective actions

Problem	Action
127.0.0.1 refused to connect	Run ./start.sh and keep its terminal open
Port 8001 already in use	Use the existing server or start with RPS_PORT=8002 ./start.sh and open that port
Not Found on a new page	Restart the correct project server and refresh the browser
Teacher has no courses	Check account reference code and course/session assignment
Temporary password missing	Admin resets the account password; copy the newly displayed value
Marks save returns conflict	Reload the course and review current marks before editing
Result is missing for a student	Check enrollment and publication; saved draft marks remain hidden
Future semester is locked	Admin updates current semester when the batch reaches it

A.19 Demonstration checklist

- Confirm CSE 2021–2022, Current semester 7 and Result processing semester 6.
- Complete the intended teacher assignments and provision teacher/student accounts.
- Use approved demonstration marks, review every result and publish only after checking.
- Verify at least one student login, then take a backup before the presentation.

APPENDIX B

DATABASE DICTIONARY AND INTERFACE REFERENCE

Table B.1 Academic registry data dictionary

Table	Key fields and constraints
departments	id PK; name unique
academic_sessions	id PK; name unique
students	id PK; student_id unique; name; email unique; department_id FK; session_id FK
teachers	id PK; teacher_id unique; name; email unique; department_id FK
courses	id PK; course_code unique; course_name; credit; semester; department_id FK; category
cohorts	id PK; department_id FK; session_id FK; current_semester; processing_semester; automatic_core; unique department/session

Table B.2 Operational data dictionary

Table	Key fields and constraints
users	id PK; username unique; hashed_password; role; reference_id; must_change_password; token_version; unique role/reference
course_assignments	id PK; course_code FK; teacher_code FK; session_id FK; unique course/teacher/session
course_students	id PK; student_id FK; course_code FK; semester; session_id FK; unique student/course/semester
course_marks	id PK; enrollment_id unique FK; four marks components; published; revision

PK means primary key and FK means foreign key. The marks enrollment foreign key is configured with database-level cascade deletion, while the API deliberately blocks operations that would delete saved results. The revision value starts at one and increases on successful marks or publication updates. Existing local databases are upgraded without fabricating grade values.

API and maintenance reference

Table B.3 Main interface routes

Route or group	Purpose and access
POST /login; GET /me	Authenticate and inspect own account
POST /change-password	Change own password and invalidate old tokens
/departments /sessions /students /teachers /courses	Admin academic maintenance
/cohorts; GET /academic-plan	Admin batch settings and semester planning
/accounts and reset-password action	Admin account provisioning and recovery
/course-assignments	Admin course/session teacher assignments
/semester-enrollments	Preview and commit batch enrollment
/bulk-import/students or teachers	Admin CSV validation and atomic import
/teacher-courses/{teacher_code}	Teacher assigned offerings
/teacher-courses/.../marks/{student_id}	Teacher revision-checked marks save
/results-review and /results-review/{id}	Admin review and revision-checked publication
/student-results/{student_id}	Admin preview or own published student results
GET /health; GET /openapi.json	Runtime status and API schema

B.1 Run the regression checks

```
Backend/.venv/bin/python -m unittest discover -s Backend/tests -v
node Frontend/tests/state.test.cjs
node --check Frontend/app.js
```

Node.js is needed for the JavaScript tests, not for normal portal use. Tokens are stored in sessionStorage for the browser tab and expire after one hour. Login limits are process-local: 20 attempts per account per minute and 100 per client per minute. Public deployment needs a shared limiter and a separate security and operations review.